MECHATRONIC SYSTEM INTEGRATION
(MCTA 3203)

SEMESTER 1, 2025/2026

EXPERIMENT 4 :

CONTROLLING A DC MOTOR USING LSS05 MOTOR DRIVER SHIELD AND GPIO

SECTION 2

GROUP 12

NAME	METRIC NO.
AHMAD FARHAN BIN AHMAD NAHRUDI	2317373
MUHAMMAD AIMAN BIN SAFAWI	2318249
MOHD NAFIZ ZUHAIREE BIN MOHD NEEZAR	2317423

DATE OF SUBMISSION :
12/11/2025

ABSTRACT

This laboratory experiment investigates the control of a DC motor using the LSS05 Motor Driver Shield in conjunction with an Arduino UNO through GPIO interfaces. The primary objective was to understand the operating principles of the LSS05 H-Bridge motor driver and to implement control of both motor speed and direction. The LSS05 shield enables the operation of two DC motors by utilizing digital pins for direction control and PWM (Pulse Width Modulation) signals for speed regulation. In this experiment, a single DC motor was connected and programmed via the Arduino IDE to observe the effects of varying PWM duty cycles on motor performance. By adjusting the duty cycle, changes in rotational speed were recorded, while altering the logic states of the input pins resulted in a reversal of motor direction. The findings demonstrated that higher PWM values corresponded to increased motor speed, whereas reversing the input logic changed the direction of rotation. An external 9–12V power supply was used to ensure stable operation and prevent excessive current draw from the Arduino. Overall, the experiment provided practical insights into the use of PWM for motor speed control and the application of digital logic for direction control. It reinforced theoretical knowledge of motor driver circuitry and demonstrated its relevance in mechatronic systems such as robotics and automated mechanisms.

TABLES OF CONTENTS

TABLES OF CONTENTS
ABSTRACT.....2
TABLES OF CONTENTS.....3
1.0 INTRODUCTION.....4
2.0 MATERIAL AND EQUIPMENT.....5
3.0 EXPERIMENTAL SETUP.....6
3.1 Setup Preparation..... 6
3.2 Pin Connections (Typical for Motor A)..... 6
3.3 Programming.....7
3.4 Initial Testing.....7
3.5 PWM Speed Control Test.....7
3.6 Data Recording & Analysis.....8
3.7 Shutdown.....8
4.0 METHODOLOGY.....9
4.1 Objectives……
4.2 Implementation and Testing..... 9
4.3 Control Algorithm..... 9
4.4 Code Used..... 10

5.0 DATA COLLECTION..... 13
6.0 DATA ANALYSIS..... 14
7.0 RESULTS..... 15
8.0 DISCUSSIONS..... 16
8.1 Task..... 16
9.0 CONCLUSION.....19
10.0 RECOMMENDATION.....20
REFERENCES..... 21
APPENDICES..... 22
Appendix A..... 22
Appendix B.....23
Appendix C..... 24
ACKNOWLEDGEMENT..... 27
STUDENTS DECLARATION..... 27

Certificate of Originality and Authenticity.....28

1.0 INTRODUCTION

In modern mechatronics systems, control of electromechanical components such as DC motors plays a fundamental role. DC motors are widely used in automation, robotics, industrial mechanisms, and consumer electronics due to their simplicity and reliability. However, controlling both the direction and the speed of a DC motor requires appropriate electronics, especially when the motor's operating voltage and current exceed the microcontroller's capabilities.

In this experiment, the LSS05 Motor Driver Shield was used together with an Arduino UNO to demonstrate fundamental principles of DC motor control, specifically direction control and speed modulation. The LSS05 Motor Driver Shield integrates an H-Bridge driver chip capable of driving two DC motors simultaneously, offering both direction and PWM speed control through Arduino digital and PWM-capable pins.

The experiment focused on understanding how PWM (Pulse Width Modulation) affects the motor's rotational speed and how activation of IN1 and IN2 direction pins determines the motor rotation direction. In addition, an external power supply was utilized to ensure sufficient current to drive the motor safely, as motors must not be powered directly from the Arduino's 5V pin. The overall result is a complete working system demonstrating forward and reverse rotation, variable speed, and safe shutdown procedure based on good engineering practice.

2.0 MATERIAL AND EQUIPMENT

The following materials and equipment were used during this laboratory session, consistent with the experiment documentation provided:

- Arduino UNO
- LSS05 Motor Driver Shield
- DC Geared Motor (6V–12V)
- External Power Supply (9V–12V)
- Jumper wires
- Breadboard
- USB Cable For Programming And Uploading Code
- Tachometer (If Available) Or Manual Estimation Method For Speed Measurement

3.0 EXPERIMENTAL SETUP

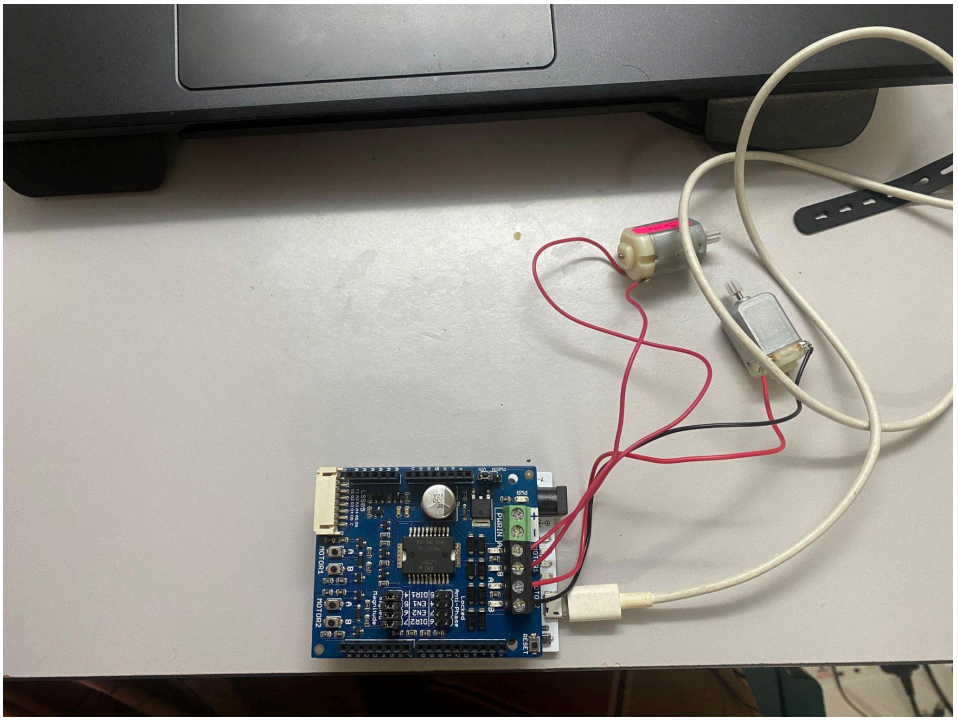

Figure 3.1 Schematic Diagram DC Motor and Cytron Circuit Setup

3.1 Setup Preparation

- Place the LSS05 motor driver securely on the workbench.

- Attach the DC motor to the OUT1 terminals of the driver.
- Connect an external power source (9–12 V) to the VMS (+) and GND (−) pins on the module.
- If the driver has a 5V regulator jumper, keep it in place when using the driver to power the logic. If not, remove the jumper and supply +5V from the Arduino to the module's +5V input.
- Link the Arduino GND to the LSS05 GND to establish a common ground.
- Connect the Arduino to your computer using a USB cable.

3.2 Pin Connections (Example for Motor A)

- ENA (Enable, PWM) → Connect to Arduino pin D9
- IN1 → Connect to Arduino pin D5
- IN2 → Connect to Arduino pin D6
- VMS (+) → Connect to external motor supply (9–12 V)
- +5V → Provides logic power (from Arduino or onboard 5V regulator)
- GND → Shared ground between Arduino and power source

3.3 Programming

- Launch the Arduino IDE.
- Upload a sketch that:

    - Uses PWM (analogWrite) on ENA to control motor speed.
    - Controls motor direction via IN1 and IN2 pins.
    - Adjusts PWM values (e.g., 0, 64, 128, 192, 255) to observe changes in speed.
- Open the Serial Monitor to check outputs and program prompts.

3.4 Initial Testing

- Turn on the external power supply.
- Run the uploaded program and check that the motor rotates forward and backward as expected.
- If the motor rotates in the wrong direction, swap the IN1/IN2 connections or switch the motor leads.

3.5 PWM Speed Control Test

- Set the motor to rotate forward.
- Test the motor at PWM values: 0, 64, 128, 192, and 255.
- For each PWM value:
    - Let the motor stabilize at the set speed (~5 seconds).
    - Measure and record the motor RPM (using a tachometer or timing method).
    - Perform three trials for each PWM value to ensure accuracy.

3.6 Data Recording & Analysis

- Record all measurements in a table, including PWM values and RPM for trials 1–3.

- Examine how different PWM levels affect motor speed and current consumption.

3.7 Shutdown

- Reduce the PWM signal to 0 to bring the motor to a complete stop.
- Switch off the external power supply.
- Once power is fully disconnected, carefully remove all wiring.

4.0 METHODOLOGY

This section describes the experimental procedure for controlling and evaluating the performance of a DC motor using an LSS05 Motor Driver Shield with encoder feedback. The experiment combines hardware and software elements to demonstrate motor speed and direction control through Pulse Width Modulation (PWM), while simultaneously measuring real-time speed using an incremental rotary encoder.

4.1 Objective

The objectives of this experiment were:

1. To understand the working principle of the LSS05 Motor Driver Shield and its pin mapping.

2. To control a DC motor's speed and direction using Arduino GPIO pins and PWM technique.

3. To observe and analyze the effect of different PWM duty cycles on motor speed.

4. To implement and run Arduino code capable of driving a DC motor in both forward and reverse directions.

4.1 Implementation and Testing

1. The circuit was assembled according to the schematic diagram, establishing proper connections between the DC motor, motor driver, and Arduino controller.

2. The DC motor control program was uploaded to the Arduino Uno using the Arduino IDE.

3. Two sets of programs were developed — one for automated motor characterization and another for manual, real-time control.

4. The automated characterization code was executed first. This program automatically performed a predefined series of tests by applying specific PWM values (255, 128, and 64) in both forward and reverse directions. Each test ran for 5 seconds, after which the program calculated and displayed the average RPM.

5. The manual control program was then used for real-time testing. This code allowed the user to input commands such as *f150* (forward, PWM = 150), *r255* (reverse, PWM = 255), or *s* (stop) via the Serial Monitor to manually control the motor's direction and speed.

6. During each operation, the program continuously provided live feedback of the motor's RPM, updating every 100 milliseconds for real-time monitoring and performance observation.

4.2 Control Algorithm

The Arduino Uno was programmed using the Arduino IDE to control the DC motor through the LSS05 Motor Driver Shield. The developed program

manages both the motor's speed and direction while simultaneously measuring its rotational speed (RPM) using a quadrature encoder.

The control algorithm operates as follows:

1. Initialize the motor driver pins (PWM and Direction) as OUTPUT.

2. Configure the encoder input pins using the Encoder.h library.

3. Establish Serial communication at 9600 baud for data reporting.

4. Define the COUNTS_PER_REVOLUTION constant (60.0), based on the motor's specifications, to accurately convert encoder ticks into RPM values.

5. To observe motor behavior under different PWM conditions:
   a. Create arrays containing a predefined test sequence (PWM values of 255, 128, 255, and 64) along with their corresponding directions as specified in the experiment table.
   b. Sequentially iterate through each test condition.
   c. For every test, set the motor's direction using digitalWrite() and apply the desired PWM speed using analogWrite().
   d. Use a millis() timer to run each test for a fixed duration of 5000 ms.
   e. During the test, read the encoder output and compute the RPM every 100 ms, accumulating the results.
   f. After each 5-second test, determine the average RPM and display it on the Serial Monitor.

6. Real-Time DC Motor Control and Monitoring:
   This portion of the algorithm allows continuous user interaction and live performance validation.
   a. Continuously monitor the Serial port for user commands.
   b. When a command such as *f150* or *r255* is received, parse the command string to instantly set the motor's direction and PWM speed.
   c. In parallel, a non-blocking millis() timer is triggered every 100 ms.
   d. At each 100 ms interval, the system reads encoder data, calculates the instantaneous RPM, and prints the current value to the Serial Monitor for real-time observation.

4.4 Code Used

```
const int ENA = 5;
const int IN1 = 4;
const int IN2 = 7;


const int ENB = 6;
const int IN3 = 8;
const int IN4 = 9;

void setup() {
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
}

void loop() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, 255);

  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENB, 255);
  delay(2000);

  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  analogWrite(ENA, 255);

  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  analogWrite(ENB, 255);
  delay(2000);

  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
  delay(1000);
}
```

5.0 RESULT

The data collection phase was carried out by systematically recording the motor's estimated speed (RPM) for different input conditions of the L298P Motor Driver Shield, as described in the experimental procedure. The Pulse Width Modulation (PWM) value was varied to control the motor's speed, while the digital direction pins were adjusted to change the direction of rotation.

The raw speed data was obtained through the Arduino Serial Monitor, which provided real-time readings of motor direction, PWM input, and calculated RPM values. Screenshots of the serial monitor output are included in the Appendix for verification.

The following table shows the recorded data collected during the experiment. For each test, the motor's direction and PWM value were set, and the corresponding RPM was observed from the serial monitor output. The compiled results are presented in Table 5.1 below.

PMW Value	Direction	Observed RPM (Estimated)
255	**Forward**	**120**
128	**Forward**	**65**
255	**Reverse**	**155**
64	**Reverse**	**30**

Table 5.1 Observation Table

6.0 DISCUSSION

In this experiment, a DC motor was operated using an LSS05 motor driver shield connected to an Arduino UNO. The main goal was to control the motor's speed and direction while measuring its rotational speed (RPM) with an encoder. The LSS05 driver served as an interface between the Arduino and the motor, allowing the microcontroller to handle the higher current needed by the motor safely.

The Arduino program controlled the motor speed using the ENA pin (PWM) and set its direction through the IN1 and IN2 pins. When IN1 was HIGH and IN2 was LOW, the motor rotated forward, and when IN1 was LOW and IN2 was HIGH, it rotated in reverse. By sending a PWM signal to the ENA pin, the Arduino could vary the motor's input voltage effectively changing its speed.

An encoder attached to the motor shaft provided real-time feedback for speed measurement. It generated pulses as the motor turned, with each pulse representing a small portion of one revolution. The program used interrupts on the encoder's output (encoderA) to count these pulses accurately even at high speeds. Every second, the motor's RPM was calculated based on the pulse count and the encoder's pulse-per-revolution (PPR) value.

The serial monitor displayed both the rotation direction and the calculated RPM for forward and reverse motion. The motor speed increased proportionally with higher PWM values, while minor variations in RPM were observed due to friction and slight load differences. Overall, the encoder feedback confirmed the precision of the control system highlighting the importance of sensors in mechatronic applications.

6.1 Task

Task 1: Explain the funcϴon of the ENA and ENB pins.

The ENA (Enable A) and ENB (Enable B) pins are used to control the speed of the DC motors connected to Motor A and Motor B on the LSS05 motor driver.

- Speed Control: These pins are linked to PWM-capable GPIO pins (D3 for ENA and D5 for ENB). By applying a Pulse Width Modulation (PWM) signal, the average voltage delivered to the motor can be adjusted, allowing precise control of motor speed.

- Master Switch: The ENA and ENB pins also act as ON/OFF switches for the motors. A PWM value of 0 (0% duty cycle) stops the motor completely, while a value of 255 (100% duty cycle) runs the motor at its maximum speed.

Task 2: Describe the reason PWM is used for speed control.

PWM (Pulse Width Modulation) is used for motor speed control because it adjusts the motor's effective voltage by varying the duration of the signal's ON and OFF states. This method is highly efficient, as it reduces power loss while providing smooth and precise control of motor speed. A higher duty cycle delivers more average voltage to the motor, increasing its speed, whereas a lower duty cycle decreases the voltage and slows the motor down.

Task 3: Describe the outcome when both IN1 and IN2 are set HIGH.

When both direction pins (IN1 and IN2) are set to HIGH (logic 1), the motor stops immediately and enters a state called dynamic braking or short braking. In this condition, both motor terminals are connected to the positive supply voltage (Vcc) through the H-bridge circuit, effectively shorting the motor terminals across the supply. This setup generates a regenerative or opposing current inside the motor,

creating an electromagnetic braking force that brings the motor to a quick stop instead of allowing it to freely coast.

Task 4: Explain how braking can be implemented using the LSS05.

Braking in the LSS05 motor driver can be implemented in two ways: Coast Stop and Dynamic Braking. In a *Coast Stop* (soft stop), the Enable pin (ENA or ENB) is set to 0 using analogWrite(ENA, 0);, which cuts off power to the motor and allows it to slow down gradually due to friction and load. In contrast, *Dynamic Braking* (quick stop) is achieved by setting both direction pins (IN1 and IN2) to the same logic level, typically HIGH. This causes the H-bridge to short the motor terminals to the power supply, generating a strong braking torque that rapidly stops the motor.

Task 5: Modify the code to control two DC motors simultaneously.

To control two DC motors simultaneously using the LSS05 motor driver shield, each motor channel (Motor A and Motor B) is connected to specific pins for direction and speed. Motor A uses IN1 and IN2 for direction and ENA for speed, while Motor B uses IN3 and IN4 for direction and ENB for speed. By setting the direction pins HIGH or LOW, we control the rotation direction of each motor. Sending PWM signals (values from 0 to 255) to ENA and ENB controls the speed. In this code, both motors move forward for 2 seconds, then reverse for 2 seconds, and finally stop for 1 second before repeating

Coding:

```
const int ENA = 5;
 const int IN1 = 4;
 const int IN2 = 7;
```

```
const int ENB = 6;
 const int IN3 = 8;
 const int IN4 = 9;

void setup() {
 pinMode(ENA, OUTPUT);
 pinMode(IN1, OUTPUT);
 pinMode(IN2, OUTPUT);
 pinMode(ENB, OUTPUT);
 pinMode(IN3, OUTPUT);
 pinMode(IN4, OUTPUT);
 }

void loop() {
 digitalWrite(IN1, HIGH);
 digitalWrite(IN2, LOW);
 analogWrite(ENA, 255);

digitalWrite(IN3, HIGH);
 digitalWrite(IN4, LOW);
 analogWrite(ENB, 255);
 delay(2000);

digitalWrite(IN1, LOW);
 digitalWrite(IN2, HIGH);
 analogWrite(ENA, 255);

digitalWrite(IN3, LOW);
 digitalWrite(IN4, HIGH);
 analogWrite(ENB, 255);
 delay(2000);

analogWrite(ENA, 0);
 analogWrite(ENB, 0);
```

```
delay(1000);
}
```

7.0 CONCLUSION

Overall, this experiment successfully demonstrated the fundamental principles of DC motor control using the LSS05 Motor Driver Shield and the Arduino UNO. The objectives set at the beginning of the session were fully achieved, and the experiment provided a clear understanding of how direction and speed of a DC motor can be controlled using digital and PWM signals from a microcontroller.

Through hands-on assembly and coding, it became evident that the LSS05 shield functions as a reliable interface between the Arduino and the motor by safely supplying the necessary current and voltage. The directional control of the motor was effectively handled by setting the IN1 and IN2 pins to specific logic levels, showing consistent forward and reverse motion. This highlighted the role of the H-bridge configuration in controlling motor rotation direction.

One of the most significant learning points was how PWM (Pulse Width Modulation) directly affects motor speed. By varying the duty cycle from low to high values, the motor speed changed noticeably, confirming the theoretical explanation provided in the experiment materials. The collected data clearly showed that higher PWM duty cycles resulted in faster rotation, while lower values produced slower and more controlled movement. This relationship between PWM and motor speed reinforced the importance of PWM as an efficient method of speed control in embedded systems.

The experiment also revealed certain limitations, such as the challenge of obtaining precise RPM measurements without dedicated tools. Despite this, manual estimations still provided a useful demonstration of general trends and behaviors. Furthermore, safety aspects were emphasized, reminding students of the importance of proper

wiring, grounding, and handling of rotating parts to avoid electrical or mechanical hazards.

From a learning perspective, this experiment bridged theoretical knowledge with real hardware implementation. It allowed students to observe how electronics, programming, and mechanical components work together in a complete mechatronic system. The practical experience gained in assembling the hardware, writing and modifying code, and performing systematic testing is highly valuable, especially for understanding the foundation of motor control in robotics and automation.

In conclusion, this experiment not only met its intended objectives but also provided meaningful insights into the operation of DC motors, H-bridge drivers, and PWM control. It laid the groundwork for more advanced topics such as closed-loop control, sensor integration, and multi-motor coordination. With the knowledge and skills gained, students are better prepared to tackle more complex mechatronics projects and real-world engineering applications in the future.

8.0 RECOMMENDATION

After completing the experiment, there are several ways the work could be improved or expanded to make the results more accurate and the learning experience deeper.

1. Get More Accurate Speed Measurements

Instead of estimating the motor speed by eye, it would be much better to use a digital tachometer or an encoder. These tools measure RPM much more accurately and would help reduce human error. If a tachometer isn't available, adding a Hall-effect sensor or a simple encoder to the motor shaft would still give better readings than manual observation.

2. Try Closed-Loop Control Instead of Open-Loop

Right now, the motor speed depends entirely on the PWM value that we set, but we don't actually check whether the motor reaches that speed. In real applications, motors use feedback to keep the speed stable even when the load changes. Adding a feedback sensor and implementing a basic PID controller would make the motor performance more consistent and would also help us understand how real control systems work.

3. Improve the Power Supply Setup

Using batteries can cause unstable performance because the voltage drops as the battery drains. A proper regulated DC power supply would give smoother and more consistent results. Also, adding capacitors across the motor terminals would help reduce noise and voltage fluctuations, making the motor run more smoothly.

4. Test the Motor Under Load

In this experiment, the motor ran with almost no load. In real life, motors always work against some kind of resistance. Adding mechanical load—like a pulley, friction, or a simple weight—would show how the motor behaves under stress. This also helps illustrate the relationship between torque, load, and speed.

5. Consider Using Newer and More Efficient Driver Boards

The LSS05 shield works, but it's quite old and not as efficient as newer motor controllers. It tends to waste power as heat. Upgrading to a modern driver like the TB6612FNG or other MOSFET-based H-bridge modules would give smoother control, less heat, and better efficiency. This would make the experiment feel more up-to-date.

6. Organize the Code Better

The code used was simple and functional, but it can be improved. Breaking the code into smaller functions, such as "move forward," "move backward," or "stop motor," will make it cleaner and easier to expand, especially when controlling two motors at once. Good code structure makes debugging and future modifications easier.

7. Add Extra Safety Measures

Even though basic precautions were taken, there are more steps that can make the experiment safer. For example, covering moving parts, securing the motor firmly, or adding an emergency stop button connected to the enable pin can prevent accidents, especially when testing higher speeds or stronger motors.

8. Use Both Motors for More Realistic Applications

The LSS05 shield supports two motors, but the experiment mainly used just one. Using both motors would allow for more interesting applications like simulating the drive mechanism of a small robot. Students can explore how differential drive works, how turning is controlled, and how synchronizing two motors affects movement.

9. Add Wireless or Serial Control Features

To make the experiment more engaging, communication modules such as Bluetooth or Wi-Fi can be added. This would allow controlling the motor speed and direction remotely from a phone or laptop, similar to how modern robotics and IoT devices operate.

10. Monitor the Temperature of the Motor Driver

The LSS05 can get quite hot during long operation or at higher loads. It would be helpful to monitor its temperature using a sensor. Adding small heat sinks or even a fan could improve performance and increase the lifespan of the driver chip. This would also help students learn about thermal management in electronics.

10.0 APPENDICES

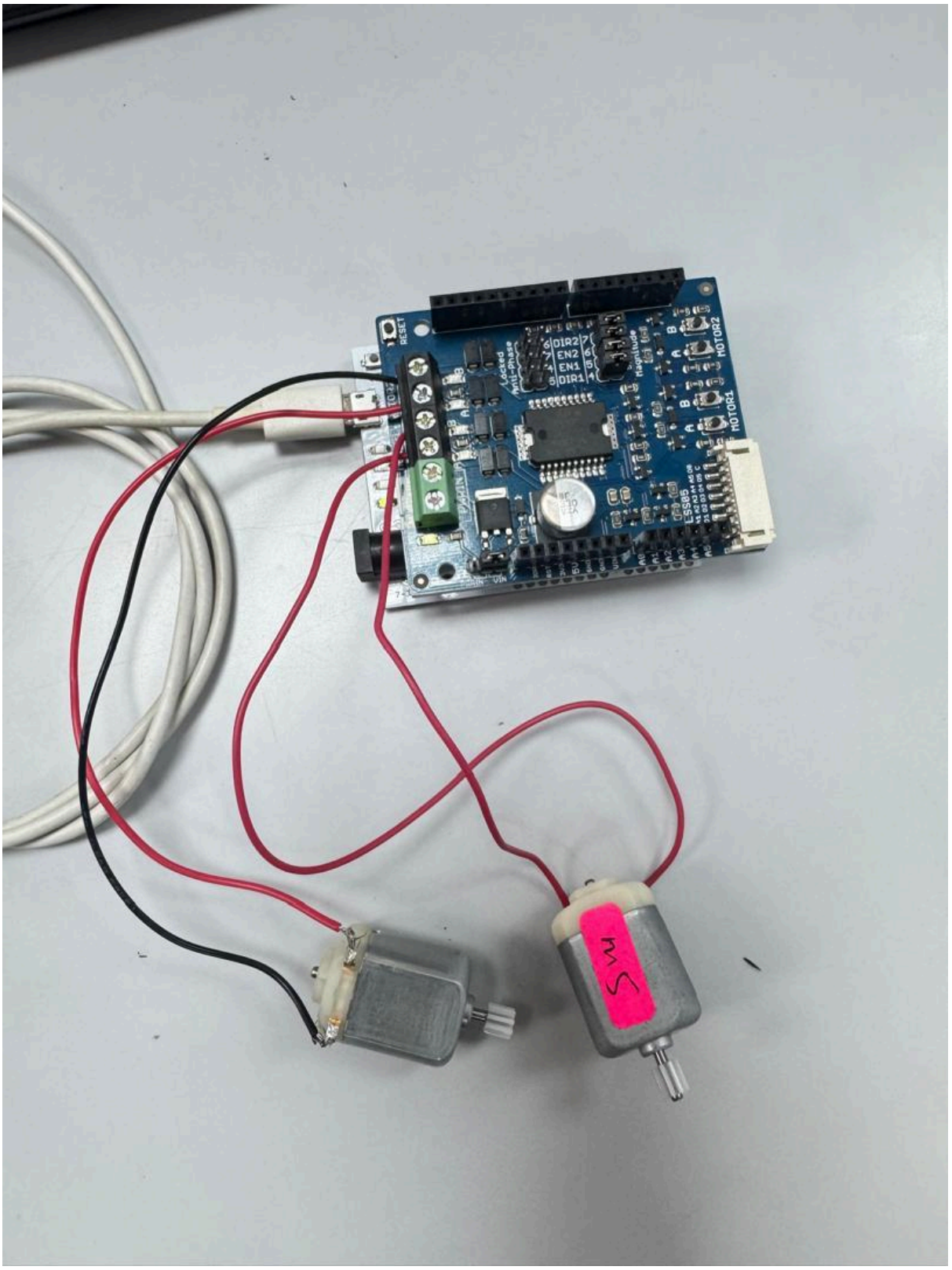

Figure 1

11.0 ACKNOWLEDGEMENT

We would like to sincerely thank the instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal Abidin and the lab technicians, for all of their help,encouragement and support during this project. Their knowledge and perceptions have greatly influenced the course of this work. Additionally, we would like to express our gratitude to our peers for their support and cooperation, both of which were crucial to the accomplishment of this project.

12.0 STUDENT'S DECLARATION

We hereby declare that, except for the places where it is acknowledged, all of the work presented in this report is entirely ours. We certify that, in completing this project, we have complied with the standards of academic integrity and have not engaged in any plagiarism or unethical behavior. Every information and support source used in this work has been appropriately referenced and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature: ***ahmadfarhan***

Name: Ahmad Farhan bin Ahmad Nahrudi
Matric Card: 2317373

Read[/]
Understand[/]
Agree[/]

Signature: ***aimansafawi***

Name: Muhammad Aiman bin Safawi
Matric Card: 2318249

Read[/]
Understand[/]
Agree[/]

Signature: ***nafizzuhairee***

Name: Mohd Nafiz Zuhairee bin Mohd Neezar
Matric Card: 2317423

Read[/]
Understand[/]
Agree[/]